

9. Working Methodology

9.1 Structure of the Pipeline

Work proceeded in five stages, each producing an artefact consumed by the next. The stages are separated by files on disk rather than by function calls, which meant every intermediate result could be inspected before the following stage ran.

<code>generate_dataset.py</code>	→	<code>dataset.csv</code>	(22,000 × 30)
<code>preprocess.py</code>	→	<code>processed_dataset.csv</code>	(22,000 × 61)
	→	<code>preprocessing_params.json</code>	(fitted constants)
<code>train.py</code>	→	<code>risk_model.pkl</code> , label encoders, <code>model_metadata.json</code>	
<code>inference.py</code>	→	<code>predict(user_input)</code>	→ prediction + SHAP
FastAPI service	→	REST endpoints + PostgreSQL persistence	

The one arrow that matters more than the others is the second output of `preprocess.py`. That file carries the constants fitted on the training data forward into serving, and its absence in an earlier iteration of this project was the source of the defects discussed in Section 22.

9.2 Stage One — Data Generation

The generator draws 22,000 records under a fixed random seed. Attributes are not sampled independently. Income is conditioned on occupation and education; expenses are conditioned on income and number of dependants; credit score is a function of debt burden, savings rate and years of experience, plus noise. The intent was to produce a dataset whose marginal and joint distributions resemble something plausible rather than a table of uniform random numbers, so that a model trained on it has a genuine structure to discover.

The risk label is derived from a weighted sum of five quantities — savings rate, credit score, debt-to-income ratio, investment experience and whether the emergency fund covers less than three months — to which Gaussian noise of standard deviation 8 is added. The resulting continuous score is cut at the 35th and 80th percentiles to give the three classes, producing a 35 / 45 / 20 split across Low, Medium and High.

The magnitude of that noise term turns out to govern everything that follows. Measured against the deterministic component, whose standard deviation is 7.64, the injected noise is slightly larger. Roughly half the variance in the label is therefore irreducible by construction, and no model — however expressive — can recover it. Section 23 derives the resulting accuracy ceiling and compares it against what the trained model achieved.

9.3 Stage Two — Cleaning, Encoding and Feature Engineering

Preprocessing runs in a fixed order, and the order is load-bearing.

Missing-value handling runs first. The script reports 7,264 missing entries in `insurance_coverage` and imputes them with the mode. That report is an artefact rather than a fact about the data: the generator emits the literal string `None` as a valid third level of the attribute, and `pandas.read_csv` treats a bare

`None` as a null by default. A third of the dataset therefore had a genuine category converted to a null and then overwritten with `Basic`. Section 12 examines the consequences. No duplicate rows were present.

Nine numeric columns are then clipped to their interquartile bounds at $1.5 \times \text{IQR}$; the columns affected most were `loan_amount` and `total_debt`, with 2,674 and 2,514 values clipped respectively.

Clipping happens *before* any ratio is derived, which is why the bounds must be carried into serving. A live request carrying an unclipped income produces a debt-to-income ratio the model never saw during training, even though the raw input is perfectly valid.

Encoding follows. Five columns have a natural order and are mapped to integers: education, income level, investment frequency, insurance coverage and shopping frequency. Six nominal columns are one-hot encoded. Because every `None` in insurance coverage had already been rewritten to `Basic`, the fitted ordinal map contains only two levels rather than three. The consequence surfaces at serving time, where a user who reports holding no insurance is encoded as the training median, and is taken up in Sections 12 and 26.

Seven features are then derived. Four are simple ratios: debt to annual income, savings ratio, expense ratio and luxury spending as a fraction of income. One is the income-to-age ratio. The remaining two are composite indices on a 0–100 scale, each a weighted sum of min–max normalised components:

Score	Components and weights
<code>behavioral_composite_score</code>	investment experience 0.35, investment frequency 0.25, insurance coverage 0.20, emergency fund 0.20
<code>financial_discipline_score</code>	savings rate 0.40, credit score 0.35, inverted debt-to-income 0.25

The min–max ranges used by these two scores are fitted across the whole training column. A single incoming request cannot recompute them, so they are persisted alongside the ordinal maps and clip bounds.

9.4 Stage Three — Model Training and Selection

The processed table is split 80/20 with stratification on the risk label under a fixed seed. Five classifiers are trained on the training partition with default hyperparameters and compared on the held-out partition: Random Forest, XGBoost, LightGBM, CatBoost and a two-hidden-layer neural network. Selection is by weighted F1, which was chosen over accuracy because the three classes are unbalanced at 35 / 45 / 20 and accuracy would reward a model that simply favoured the majority band.

The Random Forest won and was carried forward to a randomised hyperparameter search over twenty candidate configurations with three-fold cross-validation, which lifted its weighted F1 from 0.5949 to 0.6006. Five-fold cross-validation on the tuned model gives 0.5996 ± 0.0032 , indicating that the held-out figure is not an artefact of one fortunate split. Section 23 reports the full comparison.

A second Random Forest was trained to predict investment preference. It does not work, and Section 22 explains why the fault lies in how that label was constructed rather than in the model.

9.5 Stage Four — Explanation

Attributions are computed with SHAP. Because the selected model is a tree ensemble, `TreeExplainer` is used, which computes exact Shapley values for tree models in polynomial time rather than approximating them by sampling. During training, SHAP values are computed across the test partition and averaged by absolute magnitude to rank the features globally. At inference, values are computed for the single submitted row, and the ten largest by absolute value are returned to the caller.

The serving path does not preserve the sign of an attribution. It takes the absolute value and averages across the three classes before returning the ten largest, so what reaches the interface is a magnitude — how much a feature mattered — rather than a direction. Section 21.4 examines the consequences.

9.6 Stage Five — Serving

The trained artefacts are loaded once, at process start, and cached for the lifetime of the service. A request to `POST /predict` is validated against a Pydantic schema whose categorical fields are constrained to exactly the literal values seen in training, so an unrecognised category is rejected with a 422 before it can reach the model. The validated dictionary is passed to `inference.predict`, which clips it, derives the seven features, encodes the categoricals, aligns the columns to the order recorded in `feature_columns.json`, and calls the model.

`POST /assessment` performs the same work and additionally writes the submitted inputs, the prediction, and three rows of behavioural scores to the database inside a transaction. If the model call fails, the assessment row is marked `failed` rather than left dangling.

9.7 Verification of the Serving Path

Because training and serving derive features through different code, the two were checked against each other directly rather than assumed to agree. Three hundred rows were drawn at random from the raw dataset, passed through the serving preprocessor, and the resulting feature vectors compared column by column against the rows the training preprocessor had produced for the same records. All thirteen derived and encoded features agree to within floating-point rounding, on the order of $1e-16$.

A stronger check was then applied end to end. Rows from the held-out test partition were routed through the full serving path and the resulting predictions compared against those obtained by feeding the model its training-format features directly. The two agree on every row. This is the evidence that the accuracy reported in Section 23 is the accuracy a live user actually receives, which was not true of the earlier implementation.

Figure 9.1 — *Pipeline flowchart*. Five stages as boxes, artefacts as labelled arrows between them. Highlight `preprocessing_params.json` as a second output of stage two feeding stage five, since that edge is the fix described in Section 22. Place after Section 9.1.

Figure 9.2 — *Distribution of the risk score before thresholding, with the 35th and 80th percentile cuts marked*. Generate from `generate_dataset.py` by plotting the `risk_score` array. This makes the class boundaries concrete and shows the overlap that the noise term produces. Place in Section 9.2.

Table 9.1 — *Preprocessing operations in execution order*, with the column count after each. Rows: load (30), impute missing (30), clip outliers (30), ordinal-encode five columns (35), one-hot encode six nominal columns, replacing them with 25 dummies (54), derive seven features (61). The final count matches `processed_dataset.csv` . Place at the end of Section 9.3.